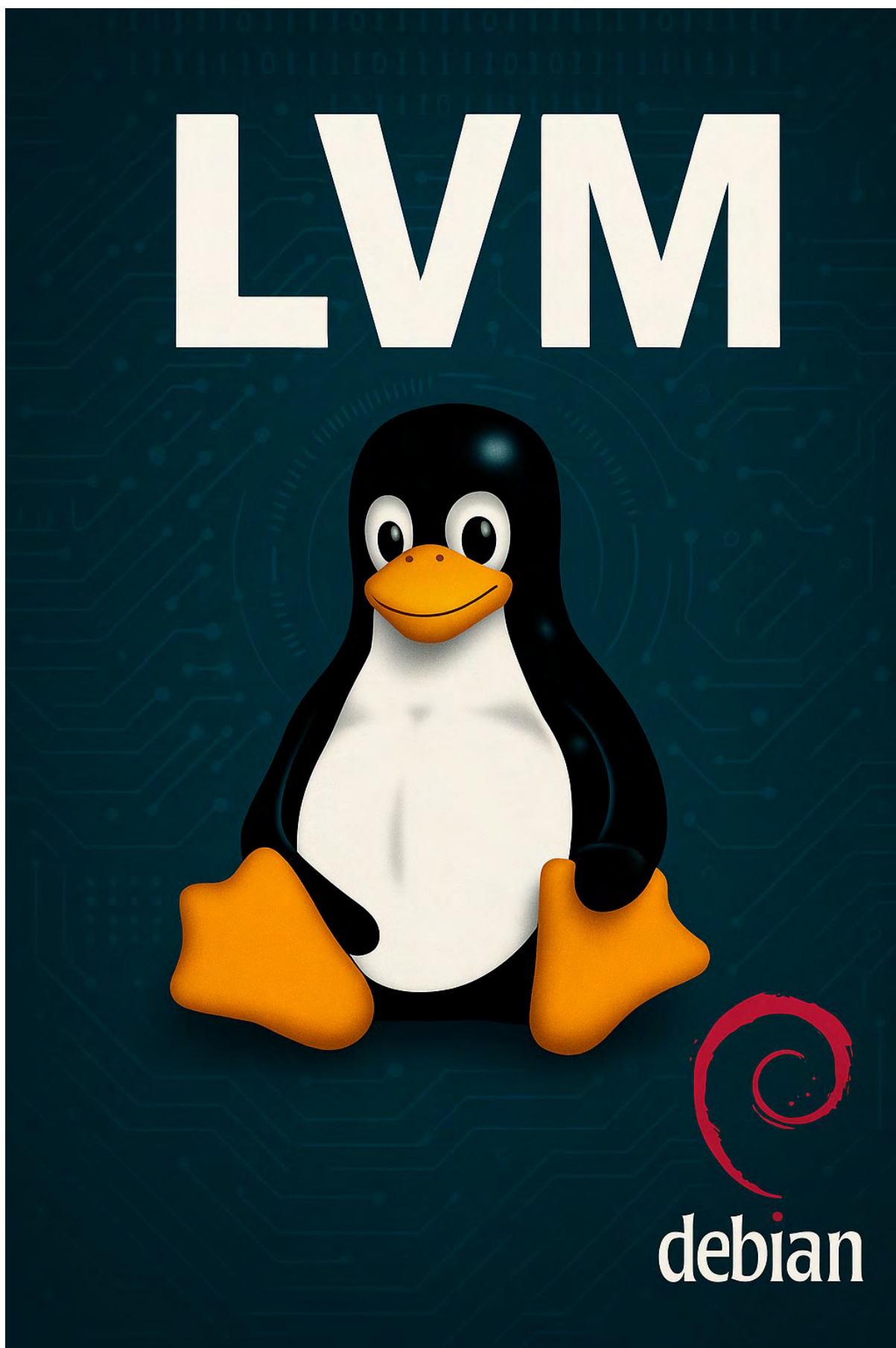




Índice del proyecto — Gestión dinámica de almacenamiento con LVM en Linux

1. [Introducción](#)
 - o ¿Qué es LVM?
 - o Ventajas frente al particionado tradicional
 2. [Entorno de trabajo](#)
 - o Software utilizado (VirtualBox, Debian/Fedora, etc.)
 - o Disposición de discos virtuales
 3. [Instalación y preparación](#)
 - o Instalación de herramientas LVM
 - o Verificación de discos disponibles
 4. [Creación de la infraestructura LVM](#)
 - o Crear Physical Volumes (PV)
 - o Crear un Volume Group (VG)
 - o Crear Logical Volumes (LV)
 5. [Montaje de volúmenes](#)
 - o Montaje y configuración en /fstab
 6. [Gestión avanzada](#)
 - o Ampliar un volumen lógico
 - o Reducir un volumen lógico (con precauciones)
 - o Snapshots: creación y restauración
 7. [Resumen y conclusiones](#)
 8. [Resumen de comandos útiles LVM](#)
-

1. Introducción

¿Qué es LVM?

LVM (Logical Volume Manager) es una herramienta de gestión de volúmenes en sistemas Linux que permite administrar el almacenamiento de manera flexible y dinámica. A diferencia del particionado tradicional, LVM permite crear, redimensionar, eliminar o mover particiones sin necesidad de reiniciar el sistema o perder datos.

Ventajas frente al particionado tradicional

- **Redimensionamiento dinámico:** Es posible aumentar o reducir el tamaño de los volúmenes lógicos en caliente (sin reiniciar).
- **Snapshots:** Permite crear capturas del estado actual de un volumen lógico para realizar backups o pruebas sin afectar el sistema principal.
- **Facilidad de gestión:** Se pueden agregar discos físicos a un grupo de volúmenes existente, extendiendo el almacenamiento **sin necesidad de reformatear**.



- **Mejor aprovechamiento del espacio:** Permite combinar múltiples discos físicos en un único grupo de almacenamiento, optimizando su uso.

Casos de uso típicos

- Entornos de servidores que requieren escalabilidad y alta disponibilidad.
- Sistemas que manejan grandes volúmenes de datos, como servidores de bases de datos o almacenamiento en red.
- Laboratorios de pruebas y entornos de desarrollo donde se necesita flexibilidad para hacer cambios frecuentes en la estructura del almacenamiento.

2. Entorno de trabajo

Software utilizado

Para este proyecto se ha utilizado el siguiente entorno:

- **Hipervisor:** VirtualBox 7.x
- **Sistema operativo:** Debian 12 (Bookworm) — También sería válido usar Fedora o Ubuntu.
- **Terminal y herramientas:** Bash, LVM2, utilidades GNU (lsblk, fdisk, df, mount, etc.)
- **Editor de texto:** nano o vim (opcional para editar fstab)
- **Sistema de archivos utilizado:** ext4

Configuración de la máquina virtual

- **CPU:** 2 núcleos
- **RAM:** 2 GB
- **Disco principal (sda):** 10 GB (instalación del sistema)
- **Disco adicional 1 (sdb):** 2 GB
- **Disco adicional 2 (sdc):** 3 GB

Los discos sdb y sdc se han añadido desde VirtualBox tras crear la máquina con el sistema base instalado. Se utilizarán para crear el grupo de volúmenes (VG).

3. Instalación y preparación

3.1 Instalación de Debian

Podemos instalar Debian ya con LVM de manera normal o cifrada(mas seguridad)

Método de particionado:

```
Guiado - utilizar todo el disco
Guiado - utilizar el disco completo y configurar LVM
Guiado - utilizar todo el disco y configurar LVM cifrado
Manual
```

Para este proyecto elegiremos la primera opción ya que nos vendrá mejor para y no hace falta complicarlo.

La segunda opción está bien si queremos aislar los archivos del usuario así aunque reinstalemos el sistema los archivos no se perderán.



La tercera opción se usa mas en servidores o entornos en producción ya que aísla mas archivos para aumentar la seguridad y estabilidad.

Esquema de particionado:

Todos los ficheros en una partición (recomendado para novatos)
 Separar la partición /home
 Separar particiones /home, /var y /tmp

3.2 Instalación de herramienta LVM

En el caso de no tener instalada la herramienta podríamos instalarla con los siguientes comandos, aunque si desde la instalación le marcamos que queremos LVM es obvio que nos lo instalará por defecto.

```
root@ProyectoLVM:~# apt update && apt install lvm2 -y
```

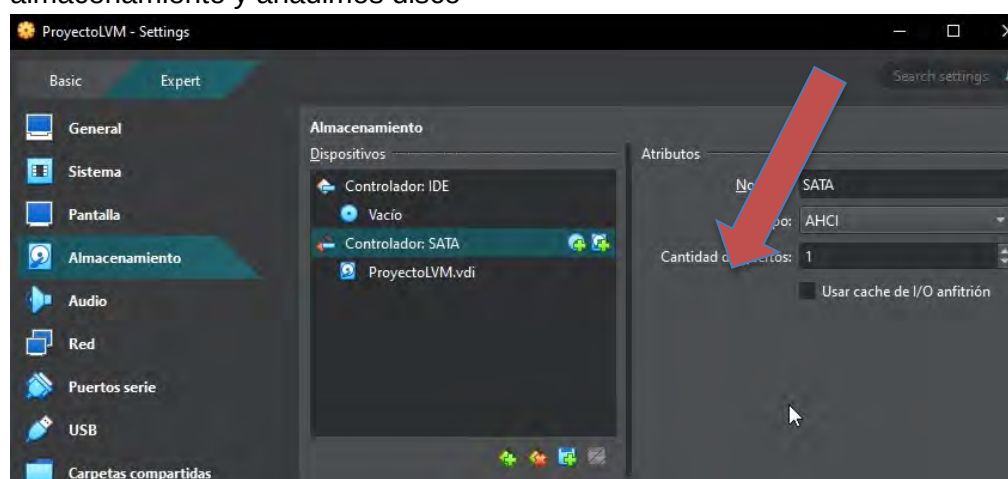
En caso de tener un sistema basado en RedHat el comando sería igual pero en vez de “apt” sería “dnf”

3.2 Verificación de discos disponibles, aun no tenemos los dos discos nuevos creados en virtualbox, se puede ver que en el TYPE se creó los lvm.

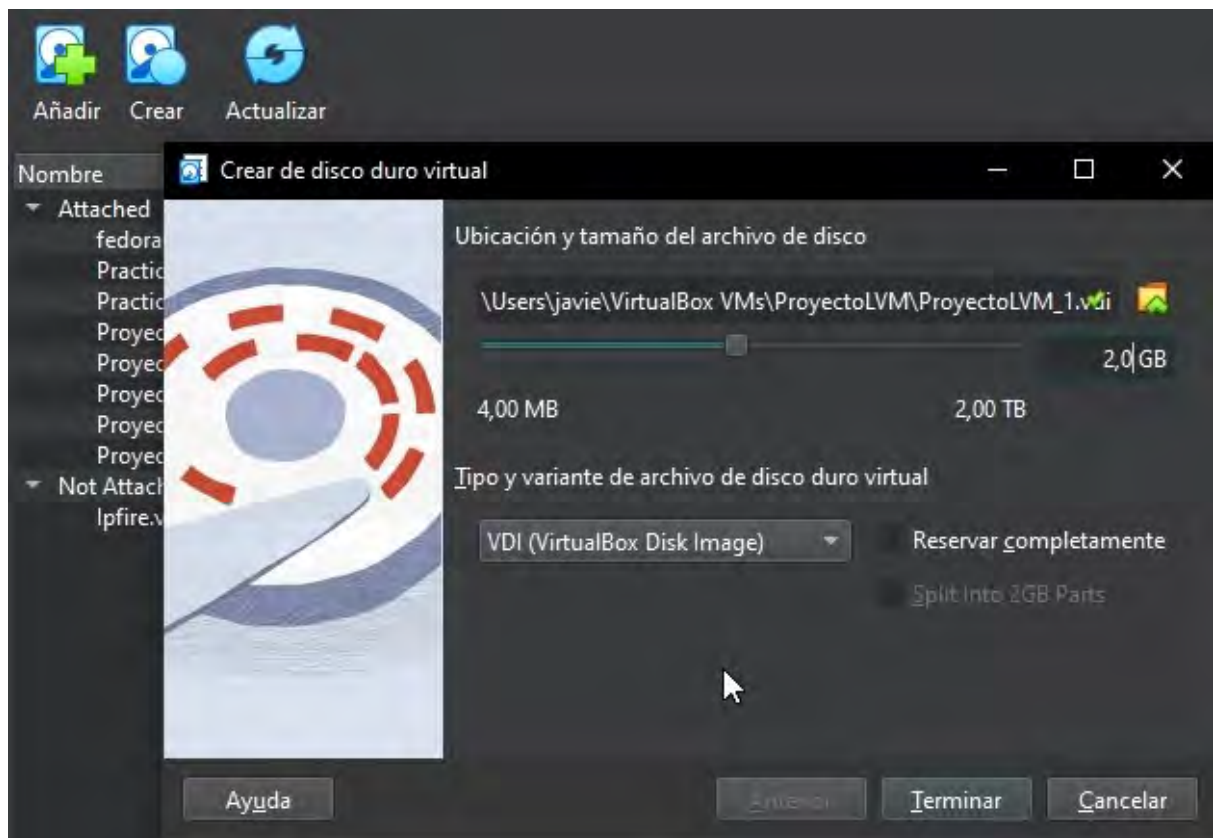
```
root@ProyectoLVM:~# lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                  8:0    0   10G  0 disk
├─sda1                              8:1    0  487M  0 part /boot
├─sda2                              8:2    0    1K  0 part
└─sda5                              8:5    0   9,5G  0 part
   ├─ProyectoLVM--vg-root          254:0    0   8,5G  0 lvm  /
   └─ProyectoLVM--vg-swap_1        254:1    0   976M  0 lvm  [SWAP]
sr0                                  11:0    1 1024M  0 rom
```

Creación de discos duros

Con la máquina apagada nos metemos en configuración de la misma, apartado almacenamiento y añadimos disco



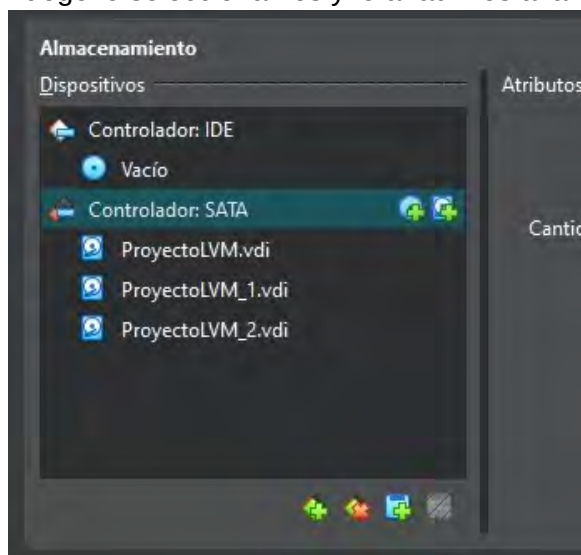
En la pantalla que nos sale le daremos a crear y configuramos el tamaño que necesitemos



Y añadimos un segundo disco

ProyectoLVM_1.vdi	2,00 GB	2,00 MB
ProyectoLVM_2.vdi	3,00 GB	2,00 MB

Luego lo seleccionamos y lo añadimos a la maquina



Una vez arranquemos podemos comprobar con “lsblk” que los discos se han añadido

```

root@ProyectoLVM:~# lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                  8:0      0   10G  0 disk
├─sda1                              8:1      0  487M  0 part /boot
├─sda2                              8:2      0    1K  0 part
├─sda5                              8:5      0   9,5G  0 part
│   └─ProyectoLVM--vg-root          254:0      0   8,5G  0 lvm  /
│       └─ProyectoLVM--vg-swap_1    254:1      0   976M  0 lvm  [SWAP]
sdb                                  8:16     0    2G  0 disk
sdc                                  8:32     0    3G  0 disk
sr0                                 11:0     1 1024M  0 rom
root@ProyectoLVM:~#

```

Nota: Cómo identificar discos y particiones en Linux

En los sistemas Linux, los discos duros y sus particiones se nombran siguiendo una lógica clara:

- sd indica que es un **dispositivo de almacenamiento tipo SATA/SCSI/USB**.
- La letra (a, b, c, etc.) indica el **orden de detección** por el sistema:
 - o sda: primer disco
 - o sdb: segundo disco
 - o sdc: tercer disco, y así sucesivamente.
- El número representa la **partición** dentro del disco:
 - o sda1: primera partición del disco sda.
 - o sdc3: tercera partición del disco sdc.

Ejemplos:

- sda1: primer disco, partición 1.
- sdb: segundo disco sin particionar.
- sdc3: tercer disco, partición 3.

4. Creación de la infraestructura LVM

En esta sección se configuran los componentes básicos de LVM: **volúmenes físicos (PV)**, **grupo de volúmenes (VG)** y **volúmenes lógicos (LV)**.

4.1 Creando Volúmenes Físicos (PV)

Ahora mismo los discos no son útiles, aunque el sistema los detecta con el lsblk no puede usarlos hasta que le creamos el volumen “pvcreate”

```

root@ProyectoLVM:~# pvcreate /dev/sdb /dev/sdc
Physical volume "/dev/sdb" successfully created.
Physical volume "/dev/sdc" successfully created.

```



Y para verificar los LVM que tenemos “pvs”

```
root@ProyectoLVM:~# pvs
PV          VG          Fmt Attr PSize PFree
/dev/sda5   ProyectoLVM-vg lvm2 a-- <9,54g 40,00m
/dev/sdb          lvm2 --- 2,00g 2,00g
/dev/sdc          lvm2 --- 3,00g 3,00g
root@ProyectoLVM:~#
```

Creando Grupo de Volúmenes (VG), el vg_datos es el nombre que le vamos a poner a ese grupo, es importante estructurar y documentar los nombres de grupos para luego no andar perdidos.

Lo que hace este comando es coger los dos discos y fusionarlo en un solo grupo

```
root@ProyectoLVM:~# vgcreate vg_datos /dev/sdb /dev/sdc
Volume group "vg_datos" successfully created
```

Otra forma de ver los nombres de los grupos de volumnes es con “vgdisplay”

Si os fijais el grupo tiene 5Gb de capacidad, 2gb de un disco y 3 del segundo.

```
root@ProyectoLVM:~# vgdisplay
--- Volume group ---
VG Name                vg_datos
System ID
Format                 lvm2
Metadata Areas         2
Metadata Sequence No   1
VG Access              read/write
VG Status              resizable
MAX LV                 0
Cur LV                0
Open LV               0
Max PV                 0
Cur PV                2
Act PV                2
VG Size                4,99 GiB
PE Size                4,00 MiB
Total PE              1278
Alloc PE / Size        0 / 0
Free PE / Size         1278 / 4,99 GiB
VG UUID                1U91Tw-2Gwv-Ko0Q-XfVP-aLpG-AefI-qYR5f8
```

También podríamos verlo con vgs, menos información.

```
root@ProyectoLVM:~# vgs
VG          #PV #LV #SN Attr   VSize VFree
ProyectoLVM-vg  1  2  0 wz--n- <9,54g 40,00m
vg_datos      2  0  0 wz--n- 4,99g 4,99g
root@ProyectoLVM:~#
```



4.3 Crear Volúmenes Lógicos (LV)

Ahora se pueden crear volúmenes lógicos sobre el grupo vg_datos. Por ejemplo, uno de 1GB para /datos1 y otro de 2GB para /datos2

Cuando creamos un LV hay que indicarle el nombre que queramos llamar al LV y asignarle el VG donde queremos crearlo..

```
root@ProyectoLVM:~# lvcreate -L 1G -n lv_datos1 vg_datos
Logical volume "lv_datos1" created.
root@ProyectoLVM:~# lvcreate -L 2G -n lv_datos2 vg_datos
Logical volume "lv_datos2" created.
```

4.4 Confirmar la estructura que hemos creado.

Como podemos ver, sdb y sdc pertenecen al grupo vg_datos y vg_datos tienen dos LVs datos1 y datos2

```
root@ProyectoLVM:~# lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                  8:0      0   10G  0 disk
├─sda1                              8:1      0   487M  0 part /boot
├─sda2                              8:2      0     1K  0 part
├─sda5                              8:5      0    9,5G  0 part
├─ProyectoLVM--vg-root             254:0      0    8,5G  0 lvm /
└─ProyectoLVM--vg-swap 1 254:1      0    976M  0 lvm [SWAP]
sdb                                  8:16      0     2G  0 disk
├─vg_datos-lv_datos1              254:2      0     1G  0 lvm
└─vg_datos-lv_datos2              254:3      0     2G  0 lvm
sdc                                  8:32      0     3G  0 disk
├─vg_datos-lv_datos2              254:3      0     2G  0 lvm
└─
sr0                                  11:0      1 1024M  0 rom
```

Ahora los LVs están listo para ser formateados, montarlos y editarlo en fstab

Formateando a la ext4

```
root@ProyectoLVM:~# mkfs.ext4 /dev/vg_datos/lv_datos1
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 262144 4k blocks and 65536 inodes
Filesystem UUID: af582734-a705-4315-a3d7-3dd60c17abf8
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376

Allocating group tables: done
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done
```

Crear puntos de montaje

```
root@ProyectoLVM:~# mkdir /mnt/datos1
root@ProyectoLVM:~# mkdir /mnt/datos2
```



Montando volúmenes

```
root@ProyectoLVM:~# mount /dev/vg_datos/lv_datos1 /mnt/datos1
root@ProyectoLVM:~# mount /dev/vg_datos/lv_datos2 /mnt/datos2
root@ProyectoLVM:~#
```

Verificando con “df -h”

```
root@ProyectoLVM:~# df -h
S.ficheros                Tamaño Usados  Disp Uso% Montado en
udev                      958M      0    958M   0% /dev
tmpfs                     197M    568K    197M   1% /run
/dev/mapper/ProyectoLVM--vg-root 8,4G    1,6G    6,4G  20% /
tmpfs                     984M      0    984M   0% /dev/shm
tmpfs                     5,0M      0     5,0M   0% /run/lock
/dev/sda1                 455M    104M    327M  25% /boot
tmpfs                     197M      0    197M   0% /run/user/0
/dev/mapper/vg_datos-lv_datos1 974M    24K    907M   1% /mnt/datos1
/dev/mapper/vg_datos-lv_datos2 2,0G    24K    1,8G   1% /mnt/datos2
root@ProyectoLVM:~# _
```

Extra: Ampliar un volumen lógico en caliente

En este ejemplo, vamos a **extender lv_datos1** de 1 GB a 1.5 GB y luego aumentar el sistema de archivos para que pueda usar ese espacio.

```
root@ProyectoLVM:~# lvextend -L +500M /dev/vg_datos/lv_datos1
Size of logical volume vg_datos/lv_datos1 changed from 1,00 GiB (256 extents) to <1,49 GiB (381 extents).
Logical volume vg_datos/lv_datos1 successfully resized.
```

Con “lvs” vemos como ha aumentado datos1 de 1G a 1,5G

¿¿¿De donde saca ese espacio extra??? Bueno si os habréis dado cuenta creamos los LVs de 1G y de 2G pero el grupo lo creamos con 5G que son los 2G de sdb y los 3G de sdc

```
root@ProyectoLVM:~# lvs
LV          VG          Attr      LSize   Pool
root        ProyectoLVM-vg -wi-ao---- 8,54g
swap_1      ProyectoLVM-vg -wi-ao---- 976,00m
lv_datos1   vg_datos     -wi-ao---- <1,49g
lv_datos2   vg_datos     -wi-ao---- 2,00g
root@ProyectoLVM:~# _
```

Redimensionar el sistema de archivos

Aunque hemos extendido el volumen, eso no quiere decir que use en ese momento lo extendido, si verificamos con “df -h” aun tiene 1G el disco1

```
/dev/mapper/vg_datos-lv_datos1 974M    24K    907M   1% /mnt/datos1
/dev/mapper/vg_datos-lv_datos2 2,0G    24K    1,8G   1% /mnt/datos2
root@ProyectoLVM:~# _
```



Con `resize2fs` extendemos el sistema de archivos hasta llenar el volumen lógico

```
root@ProyectoLVM:~# resize2fs /dev/vg_datos/lv_datos1
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/vg_datos/lv_datos1 is mounted on /mnt/datos1; on-line resizing required
old_desc_blocks = 1, new_desc_blocks = 1
The filesystem on /dev/vg_datos/lv_datos1 is now 390144 (4k) blocks long.
```

Y ahora con “`df -h`” si tenemos nuestro 1,5G

```
/dev/mapper/vg_datos-lv_datos1    1,5G    24K    1,4G    1% /mnt/datos1
/dev/mapper/vg_datos-lv_datos2    2,0G    24K    1,8G    1% /mnt/datos2
root@ProyectoLVM:~# _
```

5. Configurar el montaje automático en `/etc/fstab`

Primero obtendremos los UUID de los volúmenes (recomendado para no usar rutas directas)

```
root@ProyectoLVM:~# blkid | grep vg_datos
/dev/mapper/vg_datos-lv_datos1: UUID="af582734-a705-4315-a3d7-3dd60c17abf8" BLOCK_SIZE="4096" TYPE="ext4"
/dev/mapper/vg_datos-lv_datos2: UUID="d7d8af48-243a-459d-ac40-d3b6dcb24b79" BLOCK_SIZE="4096" TYPE="ext4"
root@ProyectoLVM:~# _
```

Editamos el archivo `/etc/fstab` con `nano` y añadimos al final.

```
GNU nano 7.2 /etc/fstab
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# systemd generates mount units based on this file, see systemd.mount(5).
# Please run 'systemctl daemon-reload' after making changes here.
#
# <file system> <mount point> <type> <options> <dump> <pass>
/dev/mapper/ProyectoLVM--vg-root / ext4 errors=remount-ro 0 1
# /boot was on /dev/sda1 during installation
UUID=67378486-e0b7-429d-89a6-863dda94abc0 /boot ext2 defaults 0 2
/dev/mapper/ProyectoLVM--vg-swap_1 none swap sw 0 0
/dev/sr0 /media/cdrom0 udf,iso9660 user,noauto 0 0
UUID=af582734-a705-4315-a3d7-8dd60c17abf8 /mnt/datos1 ext4 defaults 0 2
UUID=d7d8af48-243a-459d-ac40-d3b6dcb24b79 /mnt/datos2 ext4 defaults 0 2
```



6. Gestión avanzada

Una de las características más potentes de LVM es la capacidad de crear **snapshots**, lo que permite capturar el estado de un volumen lógico en un momento específico. Esto es útil para hacer **copias de seguridad, pruebas o restauraciones rápidas**.

6.1. ¿Qué es un snapshot?

Un **snapshot** es una copia *congelada* de un volumen lógico, creada casi al instante. LVM no duplica todo el contenido, sino que guarda las diferencias que ocurren después de su creación (sistema copy-on-write).

6.2 Creando una snapshot

Con el “-s” le indicamos que es una snapshot y con 500M le definimos cuanto cambio puede almacenar

```
root@ProyectoLVM:~# lvcreate -L 500M -s -n snap_datos1 /dev/vg_datos/lv_datos1
Logical volume "snap_datos1" created.
root@ProyectoLVM:~#
```

Con “lvs” podemos verificar

```
root@ProyectoLVM:~# lvs
LV          VG          Attr      LSize   Pool Origin         Data%  Meta%
root        ProyectoLVM-vg -wi-ao---- 8,54g
swap_1      ProyectoLVM-vg -wi-ao---- 976,00m
lv_datos1   vg_datos     owi-aos--- <1,49g
lv_datos2   vg_datos     -wi-ao---- 2,00g
snap_datos1 vg_datos     swi-a-s--- 500,00m          lv_datos1 0,01
root@ProyectoLVM:~#
```

Hagamos una prueba creando un documento de texto y metiendo cualquier cosa en el.

```
root@ProyectoLVM:~# echo "Hola que ase" | tee /mnt/datos1/prueba.txt
Hola que ase
root@ProyectoLVM:~# cat /mnt/datos1/prueba.txt
Hola que ase
root@ProyectoLVM:~# _
```

Para restaurar el volumen a su estado original usando el snapshot, primero habria que desmontar el volumen

```
root@ProyectoLVM:~# umount /mnt/datos1
root@ProyectoLVM:~# _
```

Y luego restaurarlo asi, despues habria que reiniciar para que vuelva al estado del snapshot

```
root@ProyectoLVM:~# lvconvert --merge /dev/vg_datos/snap_datos1
Delaying merge since origin is open.
Merging of snapshot vg_datos/snap_datos1 will occur on next activation of vg_datos/lv_datos1.
root@ProyectoLVM:~#
```



Antes del reinicio

```
/dev/sda1          455M    104M    327M    25% /boot
tmpfs              197M         0    197M     0% /run/user/0
/dev/mapper/vg_datos-lv_datos2 2,0G     24K    1,8G     1% /mnt/datos2
root@ProyectoLVM:~# _
```

Después

```
/dev/mapper/vg_datos-lv_datos1 1,5G     24K    1,4G     1% /mnt/datos1
/dev/mapper/vg_datos-lv_datos2 2,0G     24K    1,8G     1% /mnt/datos2
tmpfs              197M         0    197M     0% /run/user/0
root@ProyectoLVM:~# _
```

Y si hacemos “cat”

```
root@ProyectoLVM:~# cat /mnt/datos1/prueba.txt
cat: /mnt/datos1/prueba.txt: No existe el fichero o el directorio
root@ProyectoLVM:~# _
```

7. Resumen y conclusiones

Resumen del proyecto

En este proyecto se ha desplegado un entorno controlado para experimentar con **LVM (Logical Volume Manager)** en Linux, utilizando discos virtuales en una máquina creada con VirtualBox. A lo largo del proceso se ha demostrado:

- Cómo convertir discos en **volúmenes físicos (PV)**.
- Cómo agruparlos en un **grupo de volúmenes (VG)**.
- Cómo crear, montar y ampliar **volúmenes lógicos (LV)**.
- Cómo **crear snapshots** para capturar el estado de un volumen y **restaurarlo** si es necesario.
- Cómo montar automáticamente los volúmenes utilizando fstab.

Conclusiones

- LVM proporciona una **gran flexibilidad y escalabilidad** en la gestión del almacenamiento en sistemas Linux.
- Funciones como **la ampliación dinámica** y los **snapshots** son especialmente útiles en entornos reales, donde es necesario hacer cambios sin interrupciones.
- Este proyecto ha permitido **consolidar conocimientos clave de administración de discos** y sirve como base para aplicar LVM en servidores de producción, desarrollo o laboratorios.



8. Resumen de comandos útiles de LVM



Preparación

```
lsblk # Ver discos conectados
sudo apt install lvm2 # Instalar LVM en Debian/Ubuntu
```



Crear infraestructura

```
sudo pvcreate /dev/sdb /dev/sdc # Crear volúmenes físicos
sudo vgcreate vg_datos /dev/sdb /dev/sdc # Crear grupo de volúmenes
sudo lvcreate -L 1G -n lv_datos1 vg_datos # Crear volumen lógico
```



Formatear y montar

```
sudo mkfs.ext4 /dev/vg_datos/lv_datos1 # Formatear con ext4
sudo mkdir /mnt/datos1 # Crear punto de montaje
sudo mount /dev/vg_datos/lv_datos1 /mnt/datos1 # Montar
```



Gestión

```
sudo lvextend -L +500M /dev/vg_datos/lv_datos1 # Ampliar volumen
sudo resize2fs /dev/vg_datos/lv_datos1 # Ampliar sistema de archivos

sudo lvcreate -L 500M -s -n snap_datos1 /dev/vg_datos/lv_datos1 # Crear snapshot
sudo lvconvert --merge /dev/vg_datos/snap_datos1 # Restaurar snapshot
sudo lvremove /dev/vg_datos/snap_datos1 # Eliminar snapshot
```



Comprobaciones

```
sudo pvs # Ver volúmenes físicos
sudo vgs # Ver grupos de volúmenes
sudo lvs # Ver volúmenes lógicos
lsblk # Ver estructura de discos
df -h # Ver uso del sistema de archivos
```

